

CSCE 628 COMPUTATIONAL BIOLOGY · MODULE 12 · LECTURE NOTES

Pipelines and Reproducibility

An unreproducible result is not a result.

MODULE OUTCOMES

- Explain why computational biology is hard to reproduce.
- Explain reference and annotation versioning.
- Build a reproducible pipeline.
- Explain the silent failure modes.
- State what reproducibility does and does not guarantee.

1 Why it fails

- **Tool versions** — aligners, variant callers, and quantifiers change their default parameters and sometimes their algorithms between releases, and the output changes with them, usually by a little and occasionally by a lot.
- **Reference and annotation versions** — coordinates from one genome build silently mean something else in another — which is §2's subject and is the worst of these failures, because it produces no error at all.
- **Undocumented manual steps** — the file somebody edited once by hand, the sample excluded for a reason nobody wrote down, the column renamed in a spreadsheet — which are invisible in the code and decisive for the result.
- **Randomness without a recorded seed** — clustering initialisations, subsampling, bootstrap replicates, and any stochastic optimiser — all of which give different answers on every run.
- **And the environment** — library versions, locale settings (which change sort order and therefore output), and floating-point differences across platforms — which are small and occasionally decisive at a threshold. 'It worked last year' is the characteristic failure here: the tools move faster than the papers, and nothing warns you.

2 References and coordinates

A GENOMIC COORDINATE IS MEANINGLESS WITHOUT THE ASSEMBLY IT REFERS TO

Chromosome 7, position 117,120,017 is a different base in different genome builds — the assembly shifted when gaps were closed and errors corrected — **and nothing in the number itself says which build you meant.**

So mixing files from two builds produces plausible, wrong results — **no error, no warning, and the genes you report are simply the wrong ones, which is the worst possible failure mode** because nothing prompts you to check.

And lifting coordinates between builds is lossy: some regions have no equivalent in the other build, and some map to multiple places — **so the conversion itself needs reporting**, including how many positions failed to lift.

Plus annotation versions move independently of assembly versions — gene models are revised, added, and retired — **so naming 'GRCh38' is not sufficient; the annotation release matters too**, and a gene's extent can change without the assembly changing at all. **Which is Module 01 §1's check-first rule:** always 'which version of which annotation'.

3 Building one

- 1 VERSION CONTROL for the analysis code
including the small scripts, which are
exactly where the undocumented decisions live
- 2 PINNED ENVIRONMENT
exact tool versions, in a lock file or a
container image. "conda install bwa" is not a
version specification.
- 3 NAMED REFERENCE AND ANNOTATION
the assembly build, the annotation release,
and the download URL or a checksum
- 4 A WORKFLOW MANAGER
Snakemake or Nextflow: declares inputs,
outputs, and dependencies explicitly, so that
a full rerun is one command and a partial
rerun recomputes exactly what it should
- 5 SEEDS for everything stochastic, recorded
- 6 AND A TEST: run the whole pipeline on a tiny
dataset in continuous integration, so that
breakage is found when it happens rather than
at submission

THE FIRST FOUR COST ABOUT A DAY. THE LAST TWO COST AN
HOUR AND ARE SKIPPED MOST OFTEN.

A workflow manager makes a rerun one command and a partial rerun correct — which is precisely what a shell script does not give you: a script reruns everything or whatever you remember to comment out, and the second of those is how stale intermediate files end up in published results. The dependency declaration is the point, not the parallelism.

4 What it guarantees

REPRODUCIBLE MEANS SOMEBODY GETS THE SAME ANSWER FROM THE SAME DATA, WHICH IS NOT THE SAME AS BEING RIGHT

Computational reproducibility is the weakest of the three standards and the only one you fully control — same data, same code, same answer — and it is entirely within your power to deliver, which is why failing to is indefensible.

Replication is stronger: new data, the same question, the same conclusion — which is what the scientific claim actually requires and what this field's crisis was about.

And a reproducible pipeline can reproduce a confounded result perfectly, forever — Module 10 §4's batch effect survives any amount of version pinning, and so does Module 11 §2's residual population structure. Reproducibility fixes the plumbing, not the inference.

So reproducibility is necessary and not sufficient — which is exactly the shape of CSCE 632 Module 07 §3's conformance claim ('necessary, checkable, insufficient'), **and is the right way to describe a floor:** load-bearing, and not the building.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Sandve et al. — Ten simple rules for reproducible computational research (free)	§§1 and 3 — two pages, and the rules are the right ones.
The Snakemake and Nextflow documentation (free)	§3's fourth item — and the tutorials are short enough to do in an evening.
The UCSC and Ensembl genome build documentation (free)	§2 — which builds exist, what changed, and how lifting works.
Nature's reporting and code availability policies (free)	§4 — what journals now require, which is a useful minimum checklist even when you are not submitting.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Try to rerun an analysis** you did more than six months ago.
- 2 **List everything that broke**, and classify by §1's causes.
- 3 **Take one coordinate** and look it up in two builds.
- 4 **Lift a set of coordinates** and count how many fail.

- 5 **Find two public files** that use different builds of the same genome.
- 6 **Pin an environment** in a lock file or container.
- 7 **Convert one shell script** into a workflow manager pipeline.
- 8 **Delete one intermediate file** and confirm the partial rerun is correct.
- 9 **Add seeds** to every stochastic step in one analysis.
- 10 **Run the whole thing on a tiny dataset** in continuous integration.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name five causes of irreproducibility.
- 2 Which is worst, and why?
- 3 Why is a coordinate meaningless alone?
- 4 What happens when builds are mixed?
- 5 Why is lifting lossy, and what should be reported?
- 6 Why is naming the build insufficient?
- 7 Give the six pipeline requirements in order.
- 8 What does a workflow manager give you that a script does not?
- 9 Distinguish reproducibility and replication.
- 10 What can a reproducible pipeline not fix?